ABSTRACT

In modern agriculture, early detection and diagnosis of plant diseases are crucial for maintaining plant health and optimizing yield. Traditional methods for plant disease identification are often labor-intensive and require expert knowledge. This abstract presents Plant Disease Detection System using CNN leveraging machine learning techniques to enhance disease detection processes. The system utilizes a combination of image processing and machine learning algorithms to identify and classify plant diseases from leaf image of Tomato plant and utilizes react for frontend and Django rest framework for the backend and CSRF token for the authentication. Image of plant leaf is capture and processed using pre-trained model extract features and detect anomalies indicative of various diseases. This system provide detection for total of ten diseases with 74% accuracy. The system

Keyword: Plant Disease Detection System, CNN, React, Django, CSRF token, Features

ACKNOWLEGMENT

I would like to thank my project supervisors Mr. Dhiraj Jha and Mr. Pawan Niroula for their regular support and gratefulness that helped me to undertake this project. I have been greatly benefit from their regular critical reviews and inspiration throughout my work.

I would like to express my genuine thanks to the coordinator Mr. Dhiraj Jha who gave me an opportunity to undertake such a great challenging and innovative work. I am very grateful for his guidance, encouragement, understanding and insightful support in the development process.

I am also thankful to entire staff of Orchid International College for their continuous encouragement, suggestions and moral support throughout the duration of my project defense.

Aashik Thapa (26347/077)
Manish Dahal (26369/77)
Jane Dhungel (26366/077)

TABLE OF CONTENT

ABSTRACT i

ACKNOWLEGMENT ii

List of Abbreviations vii

Chapter 1: Introduction 1

1.1 Introduction 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope and Limitation 2

1.5 Development Methodology 2

Chapter 2: Background Study and Literature Review 4

2.1 Background Study 4

2.2 Literature Review 4

Chapter 3: System Analysis 7

3.1 System Analysis 7

3.1.1 Requirement Analysis 7

3.1.1 Feasibility Analysis 9

3.1.2 Analysis 10

Chapter 4: System Design 13

4.1 Design 13

4.2 Algorithm Details 16

Chapter 5: Implementation and Testing 19

5.1 Implementation 19

5.1.1 Tools Used 19

5.2 Testing 19

5.2.1 Test Cases for Unit Testing 19
5.2.2 Test Cases for Automated Testing 20
5.3 Result Analysis 21
Chapter 6: Conclusion and Future Recommendation 22
6.1 Conclusion 22
6.2 Future Recommendation 22
Reference 23
Appendix 24

List of Figures

Figure 3. 1: Use Case of Functional Requirements 7
Figure 3. 2: Work Breakdown Structure of Project 9
Figure 3. 3: Gantt chart of Project 10
Figure 3. 4: Class Diagram of System 10
Figure 3. 5: Activity Diagram of System 11
Figure 3. 6: Sequence Diagram of System 12
Figure 4. 1: Wireframe of Homepage……..……..…….……13
Figure 4. 2: Sequence Diagram of System 14
Figure 4. 3: Coponent Diagram of System 15
Figure 4. 4: Deployment Diagram of System 16
Figure 4. 4: Convolutional Neural Network Model 18

List of Tables

Table 3. 1: Descriptive Use Case of Register 8
Table 3. 2: Description Use Case for Register 8
Table 3. 3: Description Use Case for Prediction 8
Table 5. 1: Test Case of Register 19
Table 5. 2: Test Case of Login 20
Table 5. 3: Test Case of Login 20
Table 5. 4: Test Case of Automated Testing 21

List of Abbreviations

CNN - Convolutional Neural Network

ReLU - Rectified Linear Unit

CSRF - Cross-Site Request Forgery

DB - Database

SQL - Structured Query Language

VGG - Visual Geometry Group

ResNet - Residual Network

ANN - Artificial Neural Network

WBS –Work Breakdown Structure

Chapter 1: Introduction

1.1 Introduction

Agriculture complete a good portion of the global economy and plant health condition determines the sustainability in the productivity. However the tomato plants are very easy to be infected in the surrounding and disrupts the production. Early detection of the diseases in tomato plant can help to maintain the profit while minimizing the losses. Traditional methods of disease detection completely depends in the human expertise, which is costly, time consuming and prone to error.

To tackle such problem of losses and early prediction of the disease among the tomato plant, this report present the design and implementation of the Plant Disease Detection System using Convolutional Neural Networks. The machine learning technique makes the disease prediction easy and efficient for the farmers.

1.2 Problem Statement

Previously, identification of the tomato plant disease was solely dependent upon the human expertise. Farmers and experts spend considerable amount of time figuring out the disease. Some disease due to visual similarity and incorrectly diagnosed or delayed. That shorten the key issues as:

- High reliance in human judgement
- Time consuming process
- Inconsistent diagnosis

Thus, this project aims to solve such problems by creating a Plant Disease Detection system using deep learning.

1.3 Objectives

The objectives of this project are:

- To develop a system the predicts the diseases in the plant on the basis of the leaf image available
- To provide a friendly user interface to the farmers

- To integrate the system with the database for storing user data and image uploaded along with the prediction made.

1.4 Scope and Limitation

Scope:

- Detecting the disease in specific types of the plant (Tomato).
- Image conversion to gray scale and normalization and saving to csv file with respective label.
- The system will be integrated into web based platform with React for frontend and Django-REST Framework for backend.
- The training will be completed using the dataset of 3000 images in each class obtained after augmentation.

Limitation:

- The detection is limited to single plant type.
- The prediction is not absolute but is convenient.
- Low quality images may affect the model prediction
- Does not provides with the solution to the predicted disease.

1.5 Development Methodology

The development of the Plant Disease Detection System Follows an iterative and incremental approach combining the web-platform with the machine learning pipelines.

1. Dataset Selection and Preprocessing:
    - Dataset are taken form the Kaggle. The Dataset contains the 11 classes in total and each containing around 3000 images after augmentation.
    - Image was fetched and it grayscale pixel values are stored along with the Labels.
2. Model Development:
    - Designed and train the CNN model using Pytorch for feature extraction and disease classification.

- Optimized the model using the dropout, scheduler, Batch Normalization, ReLU normalization.

3. Integration with Backend and Frontend
    - Built a frontend using react and backend end using Django REST Framework.
    - Integrated model for image upload, prediction and confidence calculation using softmax.
4. Testing and Testing:
    - Split the dataset in 80 % training and 20% Testing form each class.
    - Performed hyper-parameter tuning to prevent overfitting and accuracy.

Chapter 2: Background Study and Literature Review

2.1 Background Study

Agriculture is the backbone of Nepal's economy, employing a majority of the population and contributing significantly to GDP. Maintaining plant health is critical for food security and sustainable development. Traditional methods of disease detection rely on manual inspection and chemical testing by agricultural experts, which can be time-intensive, costly, especially in rural areas. Recent advancements in computer vision and deep learning, particularly Convolutional Neural Networks (CNNs), offer efficient, automated solutions for plant disease detection.

We can use CNN to detect different kind of plant diseases analyzing its leaf images. The data set is collected form Kaggle. The model we planned predicted 10 diseases including Bacterial spot, Early Blight, Late Blight, Lead Mold, Powdery Mildew, Septoria Leaf Spot, Spider Mites, Target Spot, Tomato Mosaic Virus and Tomato Yellow Leaf Curl Virus. The images from the dataset are grayscaled and stored in the csv file pixel with the label which was later feed to the model while training. The problem of low data was solved by augmenting the images.

Some Terminology:

ReLU: ReLU is an activation function that outputs the input directly if it is positive; otherwise, it outputs zero, introducing non-linearity to the model and preventing the vanishing gradient problem.

Batch Normalization: Batch normalization normalizes the inputs of each layer during training, improving training stability, accelerating convergence, and reducing sensitivity to initialization.

2.2 Literature Review

A convolutional Neural Network (CNN) is a type of the deep learning algorithms specially used for the analyzing and processing visual data. The architecture of a CNN is composed of multiple layers, including convolutional layers, pooling layers, and fully connected layers. Advanced techniques such as batch normalization, dropout, and activation functions like ReLU further enhance the performance and generalization of CNNs. Some of the advanced

architecture of CNN includes AlexNet, VDD and ResNet which provides higher accuracy when used in the project.

Surampalli Ashok, Gemini Kishore, Velpula Rajesh, S. Suchitra, S.G. Gino Sophia, B. Pavithra(2020, June 10-12). Tomato Leaf Disease Detection Using Deep Learning Techniques has implemented plant disease detection system.They have experimented in multiple method s for disease detection like AlexNet, ANN technique and CNN (Which was the proposed method.) They have used method like Gaussian filter, Correlation for better result. The method provides them with 98.12 % accuracy rate which means the model identifies about 98.12 % of sample.

Shanthi D L, Vinutha K, Ashwini N, Saurav Vashistha(2024 May): Tomato Leaf Disease Detection Using CNN, have used a CNN model with deep architecture like AlexNet and VGGNet-16 to recognize various diseases from image datasets. The research demonstrate the use of ReLU along with total of 5 Convolutional layer and also suggest using dropout (Page-5). This model provides an accuracy of 91.2%.

Mohammed Hussein Najim, Salwa Khalid Abdulateef, Abbas Hanon Alasadi (2023 Jan): Early detection of tomato leaf diseases based on deep learning techniques, uses CNN model to detect and classify tomato plant leaf's diseases with 3 convolutional layers and 3 max pooling layers.The model was train and tested using 11000 images for 10 different disaease. Their model achieved the accuracy of 96% accuracy rate.

Mohit Agarwal, Abhishek Singh, Siddhartha Arjaria, Amit Sinha, Suneet Gupta(2020 Jan). ToLeD: Tomato Leaf Disease Detection using Convolution Neural Network created a CNN model with three max-pooling layers, three convolution layers, and two fully connected layers. The model's effectiveness was evaluated using various criteria, including training, validation, test accuracy, and the number of trainable parameters. The plant village dataset has nine disease classes and classes with healthy images. The proposed model has a 91.2% average accuracy.

Naresh K. Trivedi, Vinay Gautam, Abhineet Anand, Hani Moaiteq Aljahdali , Santos Gracia Villar, Divya Anand, Nitin Goyal, Seifedine Kadry(2021 November 30). Early Detection and Classification of Tomato Leaf Disease Using High-Performance Deep Neural Network created

CNN model with eight max-pooling layers, eight convolutional layers for plant disease detection system and the proposed model has achieved 98.49% accuracy.

Chapter 3: System Analysis

3.1 System Analysis

System analysis is e critical phase for the understanding how the system should work and determining its component and deciding how the improvement could be made to achieve the objective. The analysis if this particular project is discussed below:

3.1.1 Requirement Analysis

Requirement analysis is the process of listing out and documenting the requirements of a system or project. It is done so that the project meets the needs of the demanding party.

I. Functional Requirement

Functional requirement defines the actual outline of the system and its component. It contains the calculation, data manipulation and processing that defines what the system should do.

Functional Requirements of this system are:

- Accepting image, preprocessing the image and converting it to the acceptable format by the model i.e. (tensor).
- Making the prediction using the processed image data and storing image along with prediction

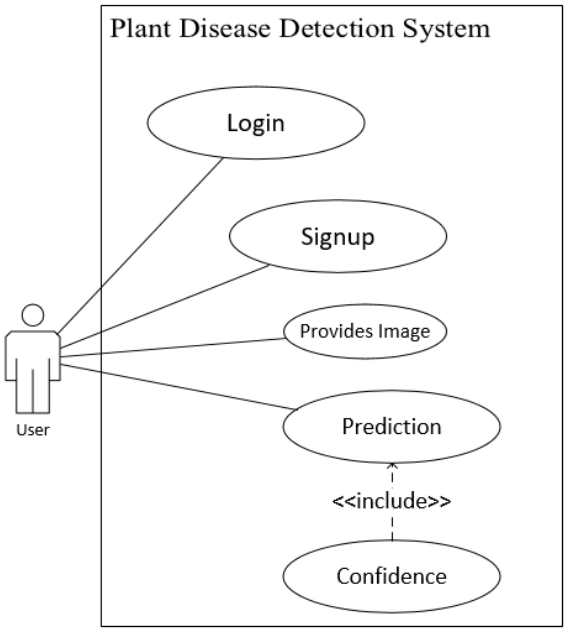


Figure 3. 1: Use Case of Functional Requirements

Table 3. 1: Descriptive Use Case of Register

Use Case ID	UC-001
Use Case Name	Register
Primary Actor	User
Secondary Actor	N/A
Description	User registration to the system
Pre-condition	All the credentials are filled with acceptable
Success Scenario	User Register Successfully
Failure Scenario	All the credential are not field.

Table 3. 2: Description Use Case for Register

Use Case ID	UC-002
Use Case Name	Login
Primary Actor	User
Secondary Actor	N/A
Description	User authentication and login to the system.
Pre-Condition	User has valid credentials.
Success Scenario	User successfully login
Failure Scenario	Invalid Credentials are submitted

Table 3. 3: Description Use Case for Prediction

Use Case ID	UC_003
Use Case Name	Prediction
Primary Actor	User
Secondary Actor	N/A
Description	Disease prediction and calculate probability
Pre-Condition	Image is passed
Success Scenario	Prediction done successfully
Failure Scenario	Connection Failure between servers.

II. Non Functional Requirement

Non-Functional requirements are the criteria for judging the operation of the system.

The non-functional requirements of this system are:

- Security: Used CSRF token for data protection along with login in signup for user.
- Usability: The system is easy to use and has user friendly UI.
- Portability: Accessible in mac, windows and large screen along small screen.

3.1.1 Feasibility Analysis

I. Technical

This system is technically feasible for the last screen device like laptop and small devices as long as they are connected in the internet.

II. Operational

The user interface is user friendly and names properly for each task. The users can easily get the prediction for the images uploaded.

III. Schedule

The project quiet did not completed as per the proposed schedule as we experienced problem regarding overfitting and authentication problem.

Mode	Task Name	Duration	Start	Finish	Predecessors	Resourc
	Project Initialization	**7 days**	**Mon 7/22/24**	**Tue 7/30/24**		
	Knowledge Gathering	6 days	Mon 7/22/24	Mon 7/29/24		
	Language Finalization	1 day	Tue 7/30/24	Tue 7/30/24	2	
	Analysis	**11 days**	**Wed 7/31/24**	**Wed 8/14/24**	**3**	
	Feasibility Study	2 days	Wed 7/31/24	Thu 8/1/24	3	
	Knowledge Analysis	9 days	Fri 8/2/24	Wed 8/14/24	5	
	Design	**15 days**	**Thu 8/15/24**	**Wed 9/4/24**	**6**	
	System Architecture Design	5 days	Thu 8/15/24	Wed 8/21/24	6	
	User Interface Design	7 days	Thu 8/22/24	Fri 8/30/24	8	
	Database Design	3 days	Mon 9/2/24	Wed 9/4/24	9	
	Implementation	**40 days**	**Thu 9/5/24**	**Wed 10/30/2**	**10**	
	Coding and integratior	30 days	Thu 9/5/24	Wed 10/16/2	10	
	Testing and debugging	10 days	Thu 10/17/24	Wed 10/30/2	12	
	Final Documentation	3 days	Thu 10/31/24	Mon 11/4/24	13	

Figure 3. 2: Work Breakdown Structure of Project

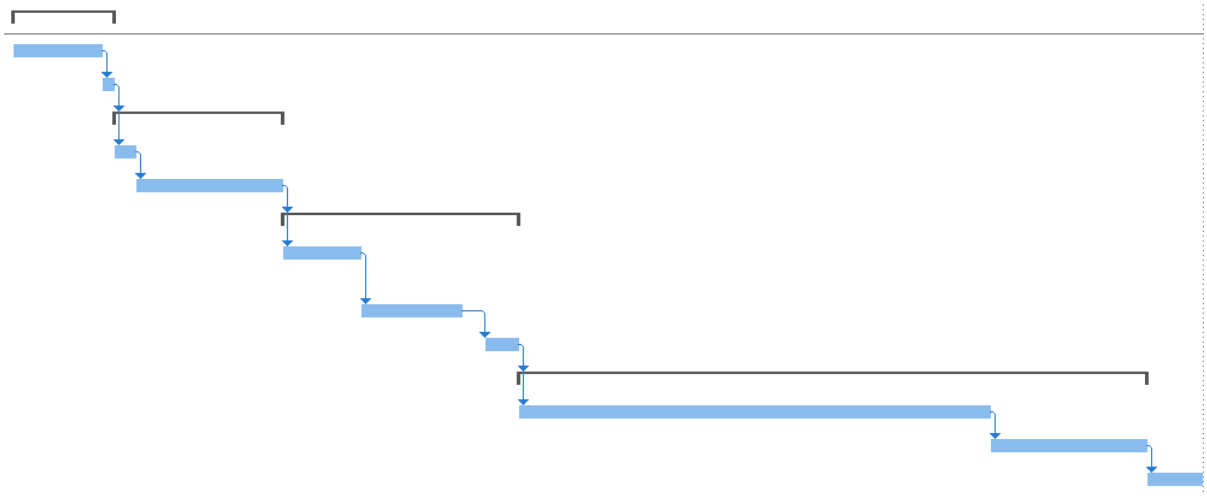


Figure 3. 3: Gantt chart of Project

3.1.2 Analysis

a. Class Diagram

The class diagram explain the core relationship between different entities. User contains attributes like user name and password and contain functionality of user login and registration. User can provide multiple images for prediction.

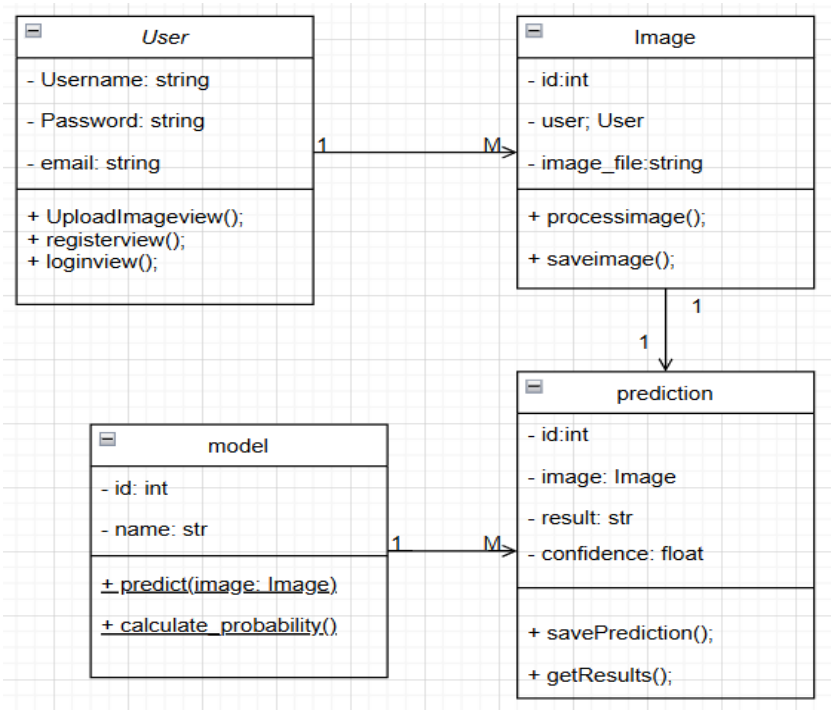


Figure 3. 4: Class Diagram of System

b. Activity Diagram

The activity diagram describes the dynamic flow of the system. User provides images to the system. As the process progress, the system process the image and utilize the CNN model and calculate the probability and predict the disease. Finally, the process concludes at the end node after display the result to its user.

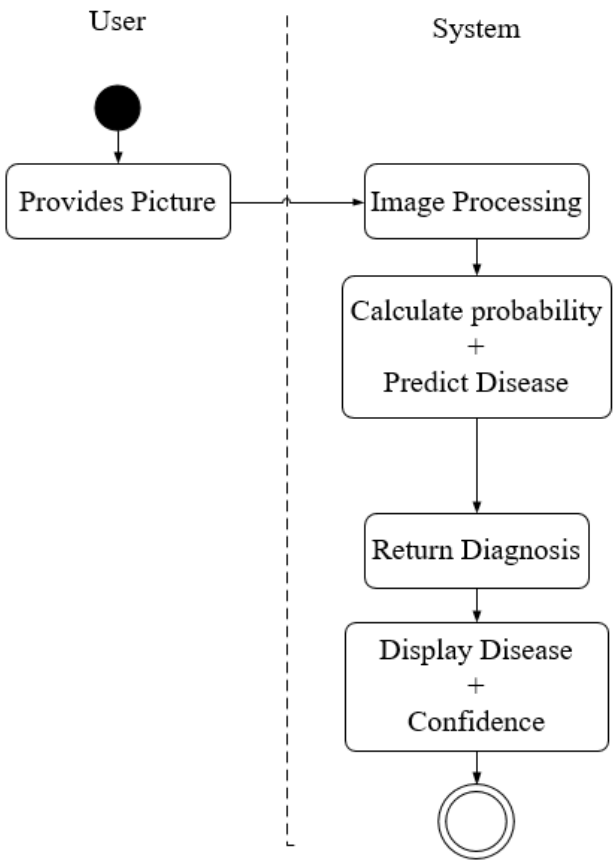


Figure 3. 5: Activity Diagram of System

a. Sequence Diagram

The sequence diagram illustrates the interaction between the user, system, and storage. Initially, the user interacts with the system by providing details for registration and login. The system stores the user details by communicating with the storage and responds by navigating the user to the homepage. After registration and login, the user uploads an image to the system. The system processes the image, predicts the disease

using a CNN model, stores the result in storage, and finally displays the prediction to the user.

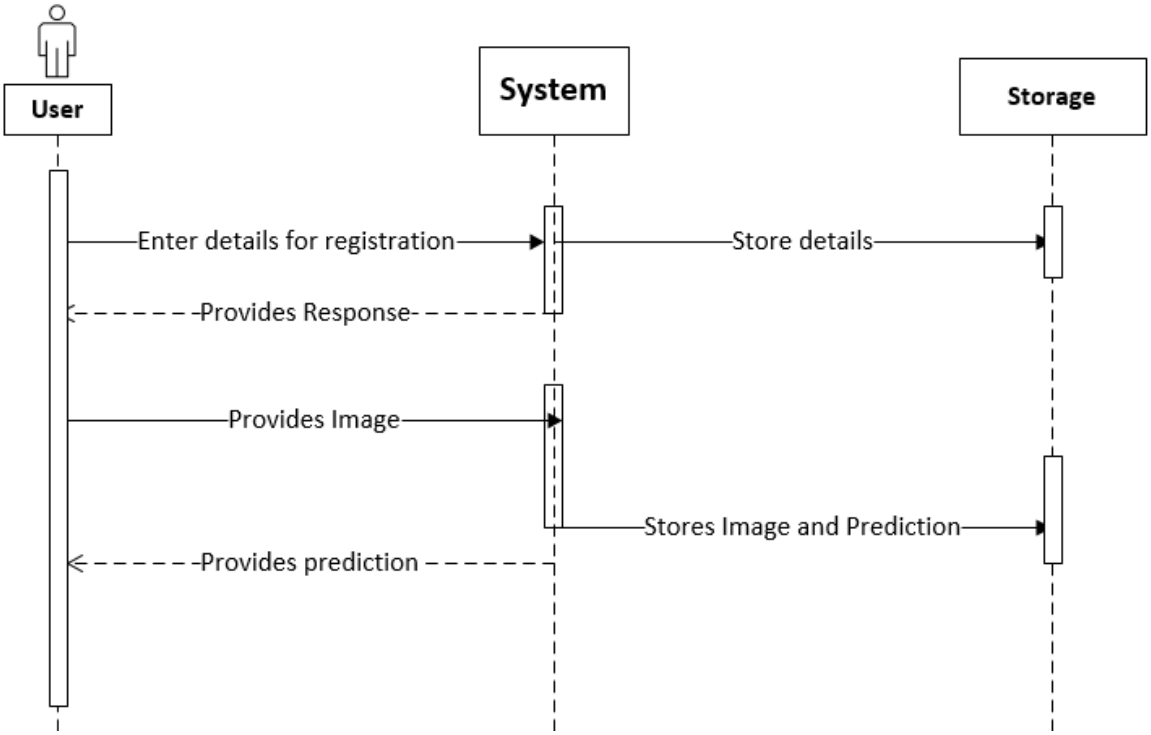


Figure 3. 6: Sequence Diagram of System

Chapter 4: System Design

4.1 Design

a. UI Wireframe

The design demonstrate the design components of the UI that includes navigation bar and image input as main component.

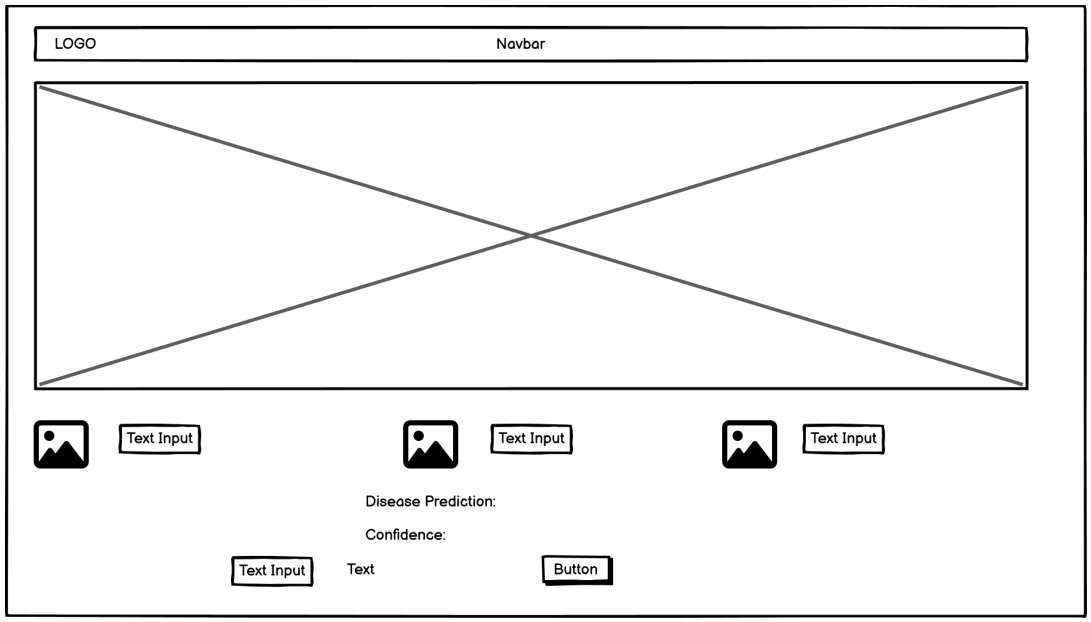


Figure 4.1: Wireframe of Homepage

b. Sequence Diagram

The sequence diagram describes detailed interaction between user, system and database. At first, the user provide details for registration if not registered. Username and password are used to login into the system. The system generates CSRF token for authentication. User upload images. The system processes the image, make a prediction and store the result in database. The system returns the prediction result to the user. When user send logout request to the system, the ending the session

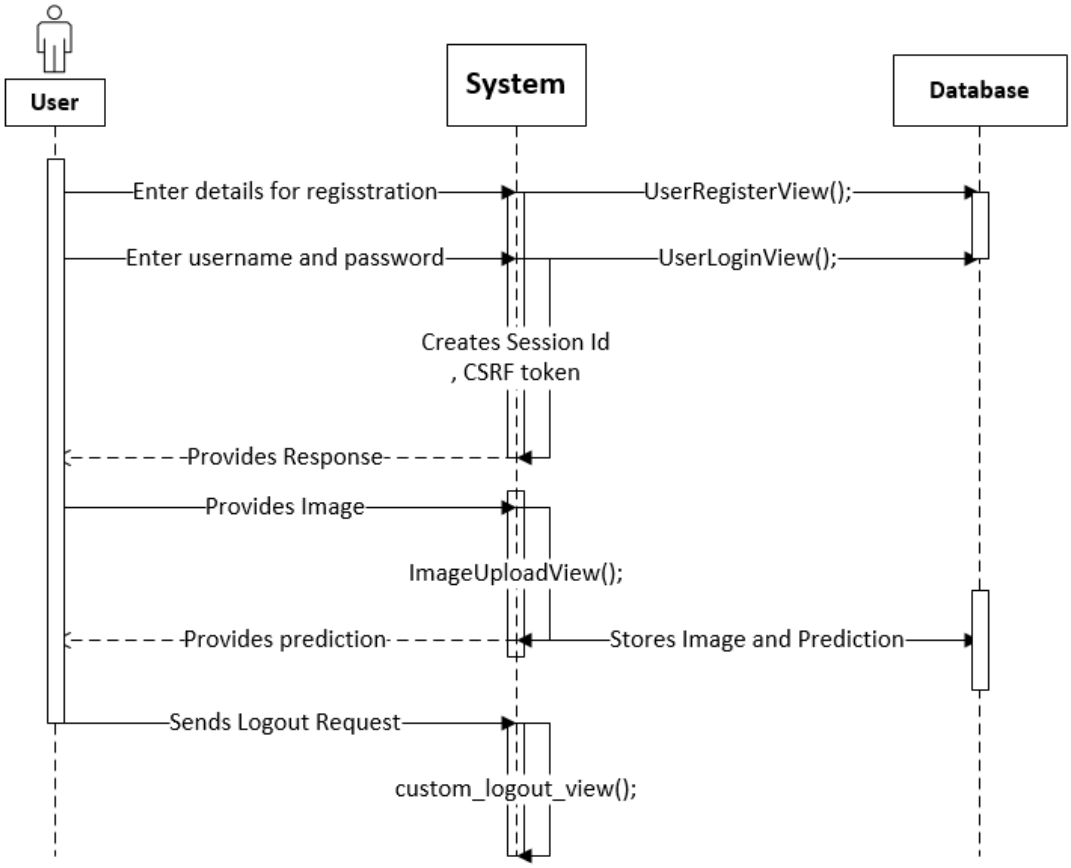


Figure 4. 2: Sequence Diagram of System

c. Component Diagram

The component diagram represents the structural organizational of the system illustrating how different components interact with each other. The system consists of three main components: Frontend, Backend, and Database. The Frontend includes the User and Image components, where users interact with the system by registering, logging in, and uploading images. These images are send to the backend that contain the Image View and Model components. Image view interact with model that provide prediction of the image. The predicted result is stored in database. The frontend retrieve and display the result to user.

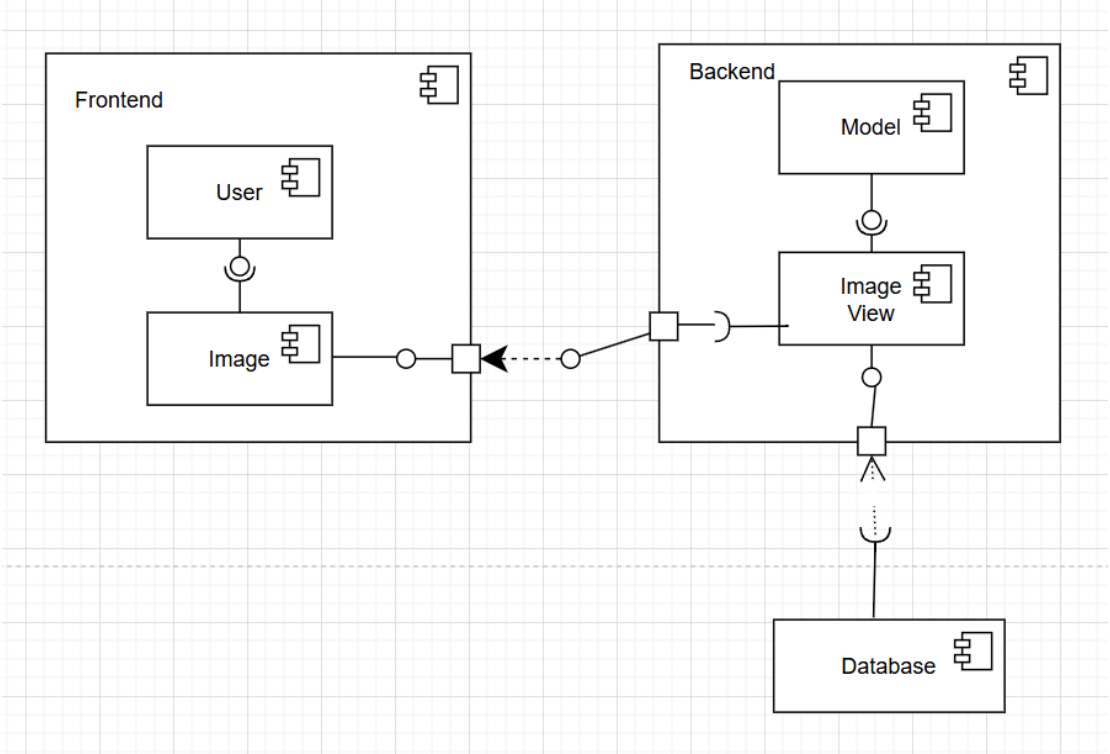


Figure 4. 3: Component Diagram of System

d. Deployment Diagram

The deployment diagram represent the physical architecture of a system showing how different nodes communicate with each other in real world environment. Frontend node interact with Application server to access the services meanwhile the application server uses database server and machine learning to provide the services to the user.

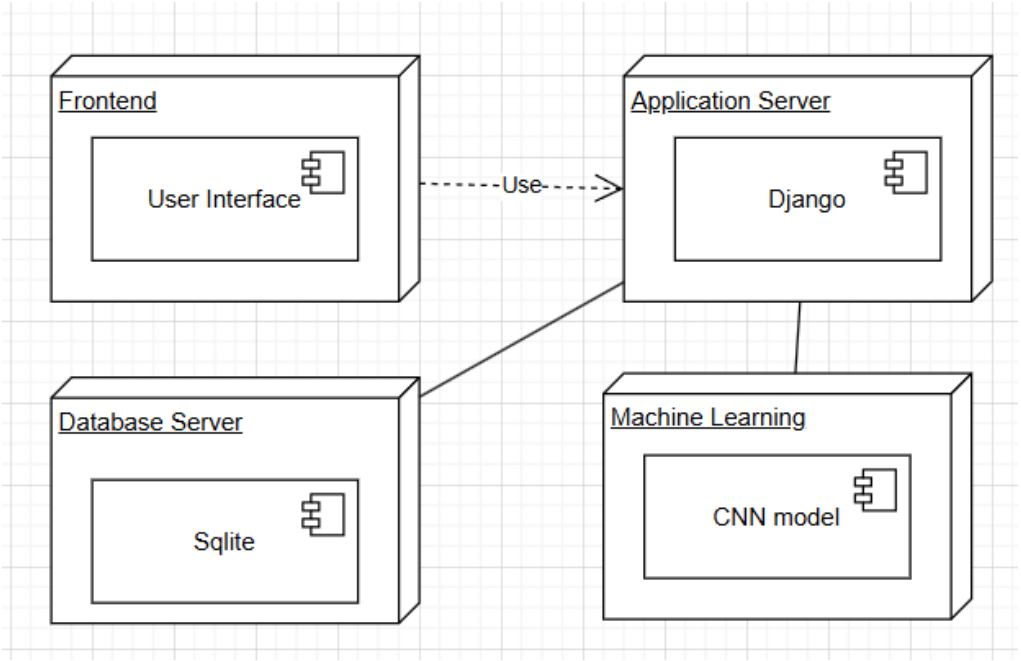


Figure 4. 4: Deployment Diagram of System

4.2 Algorithm Details

Here the algorithm used is CNN (Convolutional Neural Network). The presented algorithm is a Simple CNN model tuned through multiple trails.

The model process grayscale images (1 channel) of size 128*128 into one of the 11 classes. The architecture follows a sequence of convolutional, activation, normalization, pooling and fully connected layer.

Convolutional Layer:

The kernel glide over the leaf image and each step of glide is determined by the “stride”.

Value after convolution layer in position x, y when k kernel is applied is given by:

$$O(x,y) = \sum_{i=o}^{k-1} \sum_{j=v}^{k-1} \mathrm{I}(\mathrm{x}+\mathrm{i}, \mathrm{y}+\mathrm{j}) \cdot \mathrm{K}(\mathrm{i}, \mathrm{j})$$

And output size is given by:

Output_Size=floor_value [((Input size + 2*Padding – Kernel size) / Stride) + 1]

Batch Normalization:

It normalizes the output of the convolutional layer to stabilize the learning and improve convergence.

ReLU Activation:

Applies non-linearity, allowing the model to capture complex pattern.

Max pooling layer:

In this layer the kernel slide over the tomato leaf image determined by the stride resulting the max values corresponding to the kernel position. Size is determined by the formula:

Output Size = ((Input size – Kernel size) / Stride) + 1

Flatten:

It converts the extracted features from the leaf image in the form (batch size, channel, height, width) into (batch Size, channel * height * width) so that it can be passed to the Fully Connected Layer. Simply speaking, it convert the data with height and width into vector (single dimension).

Fully Connected layer:

This layer maps the features extracted from the tomato leaf to the final output. It connects all the features from the previous layer to learn relationship between the features. The output is the score that identifies the cross ponding class.

Dropout Layer:

It randomly set the neurons value to 0, reducing overfitting.

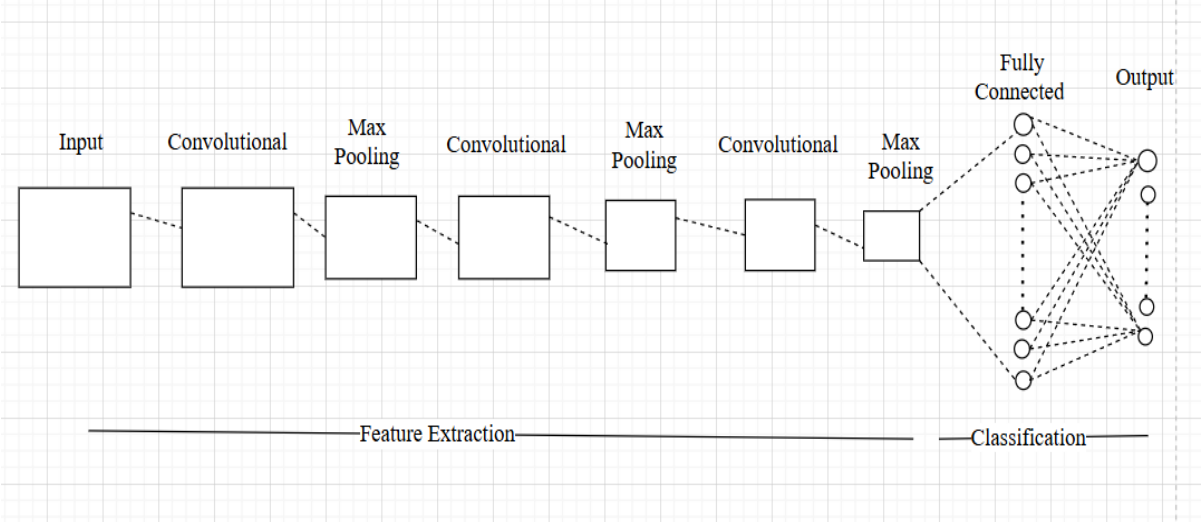


Figure 4. 1: Convolutional Neural Network Model

Chapter 5: Implementation and Testing

5.1 Implementation

5.1.1 Tools Used

Frontend: React, TailwindCSS

Backend: Django-Rest-Framework

Database: DBSqlite

Testing: Postman, Selenium

5.2 Testing

5.2.1 Test Cases for Unit Testing

For the sake of the unit testing we have use the PostMan. The testing includes the success test and invalid credential test in both registration and login.

Table 5. 1: Test Case of Register

Test Case ID	TC_001
Test Case Name	Registration Model
Objectives	To verify that the system correctly register the new user
Pre-Condition	• The server is up and running • The user provide with username email and password
Test Data	Email: aashik@gmail.com Username: aashik Password: aashik
Actual Result	User Register Successfully
Post-Condition	User Details stored in the database
Status	Pass

Table 5. 2: Test Case of Login

Test Case ID	TC_002
Test Case Name	Login module
Objectives	To verify the system authenticate the user
Pre-Condition	• The server is up and running • The username and password are previously registered
Test Data	Username: jane Password: jane
Actual Result	Login Successful
Post-Condition	New session key stored in Db_sqlite
Status	Pass

Table 5. 3: Test Case of Login

Test Case ID	TC_003
Test Case Name	Login module
Objectives	To verify the system restrict the unauthorized user
Pre-Condition	• The server is up and running • The username and password not are previously registered
Test Data	Username: jane Password: janae
Actual Result	Invalid Username or password
Post-Condition	No new session stored in Db_sqlite
Status	Pass

5.2.2 Test Cases for Automated Testing

For automated testing, Selenium is used. It involves login and feeding image to the system and getting prediction and confidence.

Table 5. 4: Test Case of Automated Testing

Test Case ID	TC_004
Test Case Name	System Testing
Objectives	To verify the system work properly as whole
Pre-Condition	• The server is up and running • The username and password not are previously registered • Image is provided for testing
Test Data	Username: jane Password: jane Image: Image path
Actual Result	Prediction and Confidence
Post-Condition	Image, Prediction and Confidence are stored
Status	Pass

5.3 Result Analysis

The system successfully passes all test cases, confirming that it functions as expected. Regarding prediction accuracy, the system achieves 74%, indicating a reasonably effective performance. Accuracy is computed using the formula:

Accuracy = Correct prediction / total prediction * 100

Several factors influence the model's accuracy along with image quality. The accuracy of the system is not very high thus may produce an incorrect prediction. Environmental conditions such as humidity, temperature, and altitude can significantly impact predictions. Since different regions have varying climatic conditions, the prevalence and characteristics of plant diseases may also vary. These variations should be considered to enhance model adaptability and overall effectiveness.

Chapter 6: Conclusion and Future Recommendation

6.1 Conclusion

The plant disease detection system predicts plant leaf diseases based on images from the dataset. It offers an interactive user interface, allowing farmers to upload images and receive disease predictions easily. The system ensures and maintains consistency in identifying different diseases. To enhance usability, the system devised to be user friendly and easy to use as instructions are simple to understand. The system stores images and their prediction for future reference and upgrades. The data stored is user based and images and prediction store are associated with particular user.

6.2 Future Recommendation

For better result and usability, things that can be added are:

- Use of the actual set of data instead of the augmented dataset.
- Use of pre-defined architecture could be fruit full giving better accuracy.
- Integration of real-time disease detection and diagnosis.
- Expansion of disease classifications

Reference

1. S. Ashok, G. Kishore, V. Rajesh, S. Suchitra, S. G. G. Sophia and B. Pavithra, "Tomato Leaf Disease Detection Using Deep Learning Techniques," 2020 5th International Conference on Communication and Electronics Systems (ICCES),Coimbatore,India,2020,pp.979-983 https://ieeexplore.ieee.org/abstract/document/9137986
2. Shanthi D L, Vinutha K, Ashwini N, Saurav Vashistha "Tomato Leaf Disease Detection Using CNN" 2024 May BMS Institute of Technology and Management, Yelahanka, Bengaluru-560064, India PWC AC, Kolkata, India https://www.sciencedirect.com/science/article/pii/S1877050924009608?via%3Dihub
3. Najim, Mohammed & Abdulateef, Salwa & Alasadi, Abbas. (2024). Early detection of tomato leaf diseases based on deep learning techniques. IAES International Journal of Artificial Intelligence (IJ-AI). 13. 509. 10.11591/ijai.v13.i1.pp509-515. https://www.researchgate.net/publication/378637904_Early_detection_of_tomato_leaf_diseases_based_on_deep_learning_techniques
4. Agarwal, Mohit & Singh, Abhishek & Arjaria, Siddhartha & Sinha, Amit & Gupta, Suneet. (2020). ToLeD: Tomato Leaf Disease Detection using Convolution Neural Network. Procedia Computer Science. 167. 293-301. 10.1016/j.procs.2020.03.225. https://www.researchgate.net/publication/340705523_ToLeD_Tomato_Leaf_Disease_Detection_using_Convolution_Neural_Network
5. Trivedi, N.K.; Gautam, V.; Anand, A.; Aljahdali, H.M.; Villar, S.G.; Anand, D.; Goyal, N.; Kadry, S. Early Detection and Classification of Tomato Leaf Disease Using High-Performance Deep Neural Network. Sensors 2021, 21, 7987. https://doi.org/10.3390/s21237987 https://www.mdpi.com/1424-8220/21/23/7987x

Appendix

Preprocessing Code:

```
Train_Dataset=r"C:\Users\AASHIK\Desktop\project model handeling\1\Our image samples"
```

```
pixels=np.array(['pixel_{:04d}'.format(x) for x in range(16384)])
label_dict = {}
Train_Dataset=r"C:\Users\AASHIK\Desktop\project model handeling\1\Our image samples"
for label_index,tomato_images_folder in enumerate(sorted(os.listdir(Train_Dataset))):
    label_dict[tomato_images_folder] = label_index
    Disease_folder_path=os.path.join(Train_Dataset,tomato_images_folder)
    Images_DF=pd.DataFrame(columns=pixels);
    output_csv="train1.csv"
    file_exists = os.path.exists(output_csv)
    for x,tomato_leaves_images in enumerate(os.listdir(Disease_folder_path)):
        Timages=Image.open(os.path.join(Disease_folder_path,tomato_leaves_images))
        new_size=(128,128)
        resizeT=Timages.resize(new_size)
        T_Grayscale=np.array(resizeT.convert("L"))
        Create_Serie=pd.Series(T_Grayscale.flatten(),index=pixels)
        Images_DF.loc[x]=Create_Serie
    Images_DF=(Images_DF).astype(np.uint8)
    Images_DF['Label']=  label_index
    if file_exists:
        Images_DF.to_csv(output_csv, index=False, mode='a', header=False)
    else:
        Images_DF.to_csv(output_csv, index=False, mode='w', header=True)
        file_exists = True
```

Model:

```
class DiseaseCNN(nn.Module):
    def __init__(self):
        super(DiseaseCNN, self).__init__()

        # Define layers using Sequential
        self.model = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=2, stride=1, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2, padding=0),

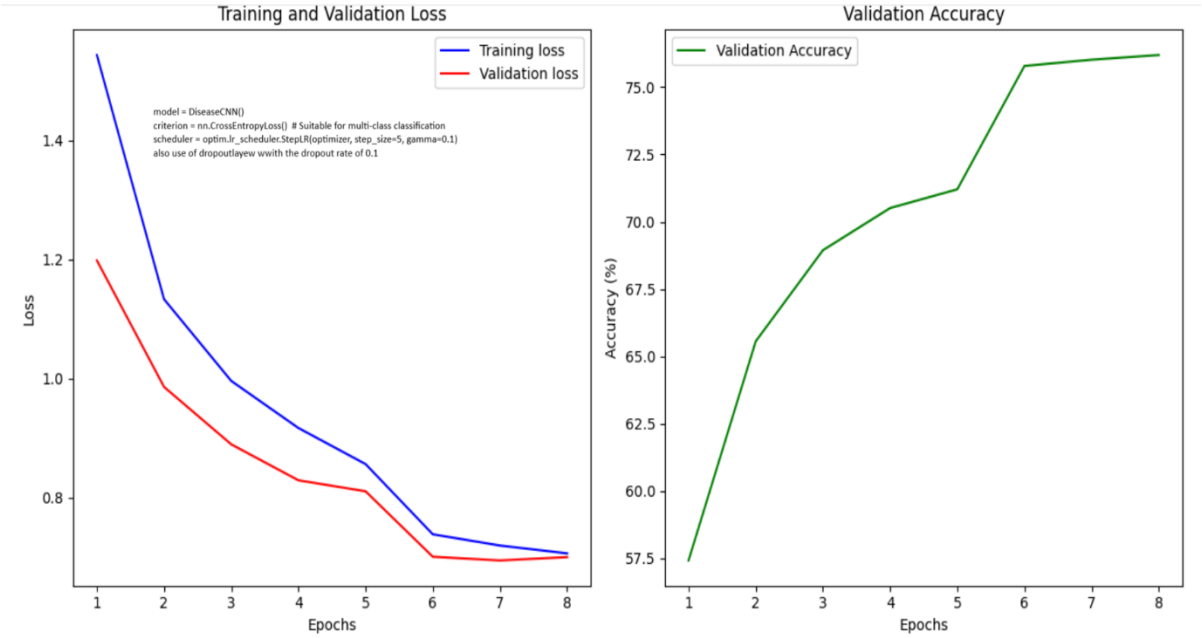
            nn.Conv2d(32, 64, kernel_size=2, stride=1, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2, padding=0),

            nn.Conv2d(64, 128, kernel_size=2, stride=1, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2, padding=0),

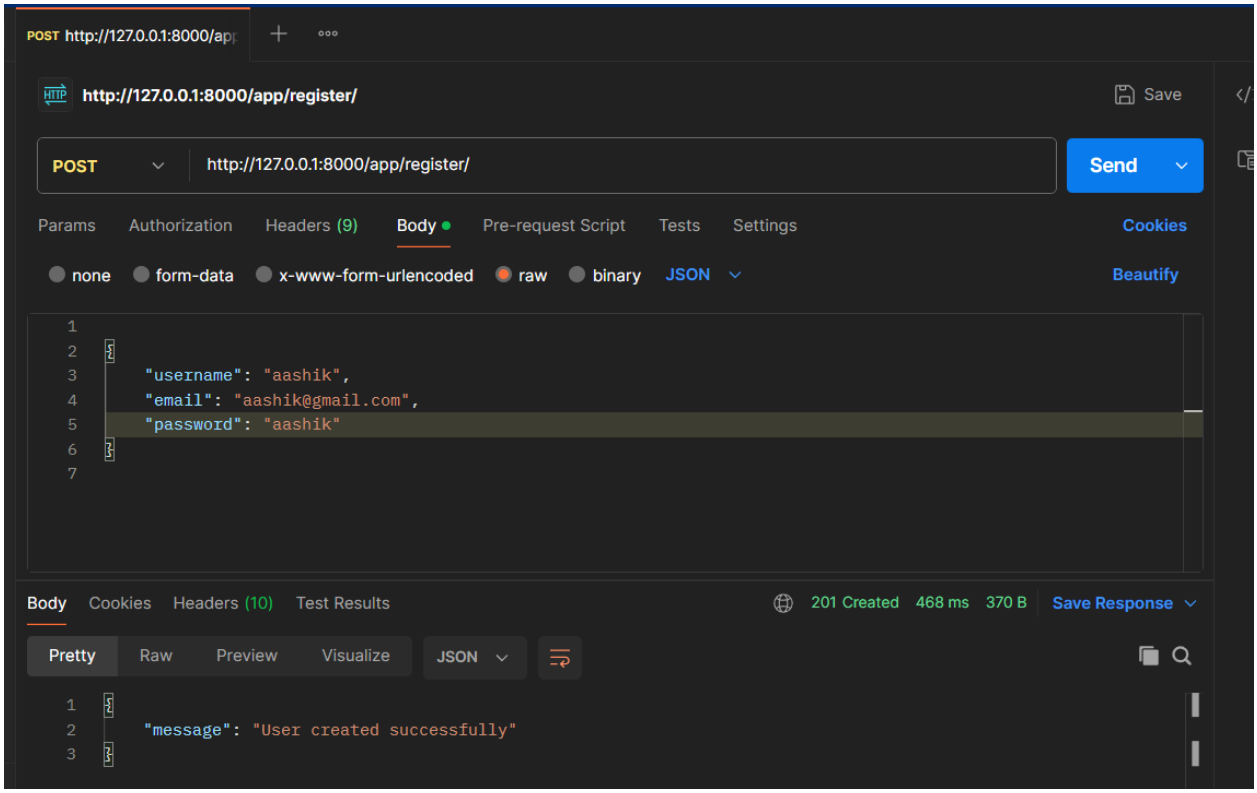
            nn.Flatten(),
            nn.Linear(128 * 16 * 16, 512),
            nn.Dropout(0.1),
            nn.ReLU(),
            nn.Linear(512, 11)
        )

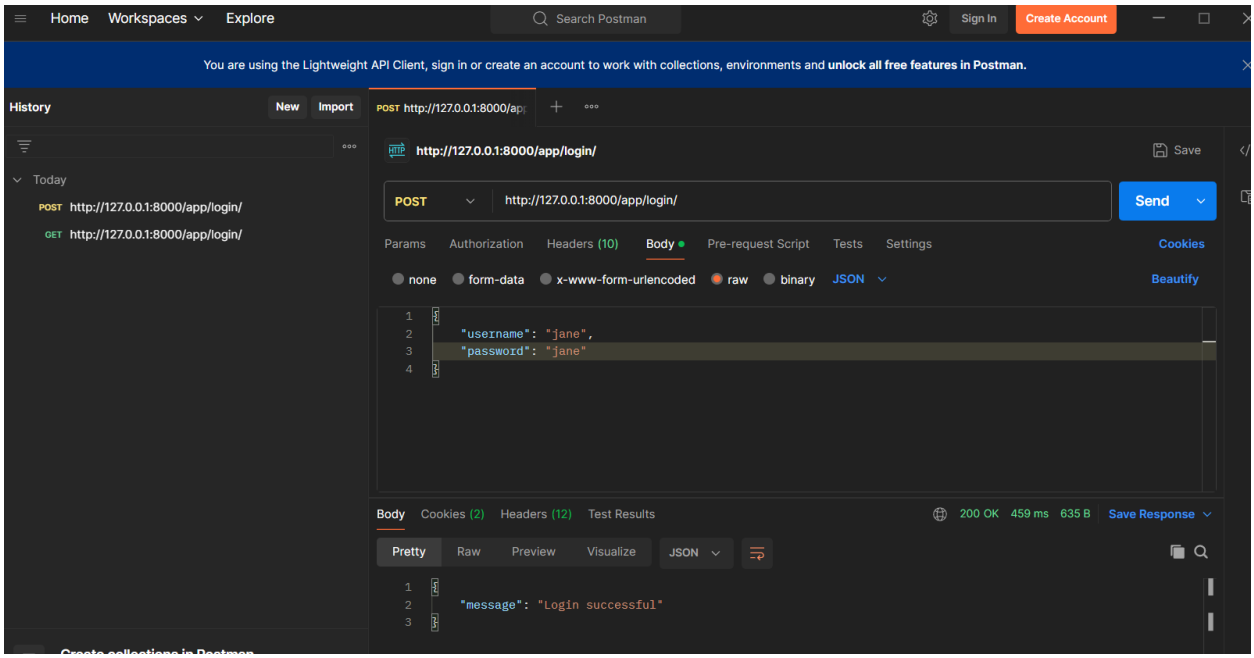
    def forward(self, x):
        return self.model(x)
```

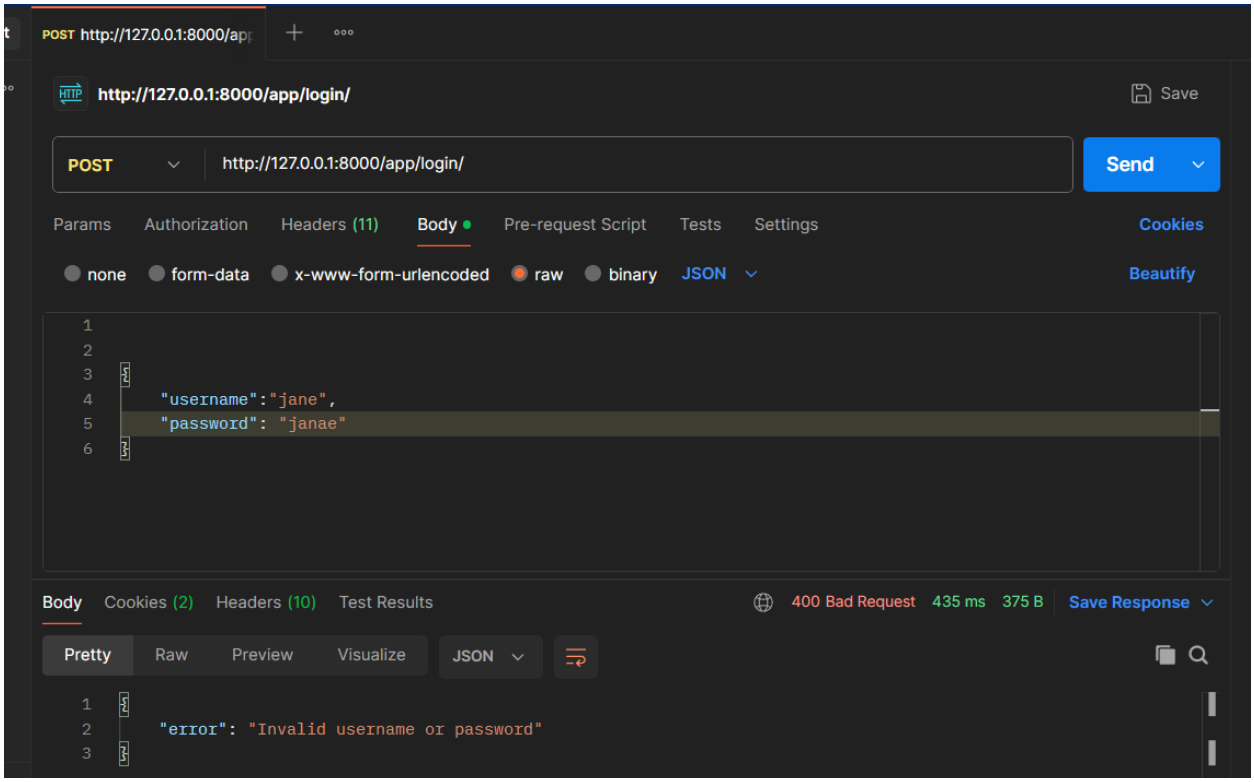
Accuracy and Loss Graph:



Testing:


POST http://127.0.0.1:8000/ap
http://127.0.0.1:8000/app/register/
Save
POST
http://127.0.0.1:8000/app/register/
Send
Params
Authorization
Headers (9)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
{
"username": "aashik",
"email": "aashik@gmail.com",
"password": "aashik"
}
Body
Cookies
Headers (10)
Test Results
201 Created
468 ms
370 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"message": "User created successfully"
}


Home
Workspaces
Explore
Search Postman
Sign In
Create Account
You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.
History
New
Import
Today
POST http://127.0.0.1:8000/app/login/
GET http://127.0.0.1:8000/app/login/
http://127.0.0.1:8000/app/login/
Save
POST
http://127.0.0.1:8000/app/login/
Send
Params
Authorization
Headers (10)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
"username": "jane",
"password": "jane"
Body
Cookies (2)
Headers (12)
Test Results
200 OK
459 ms
635 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
"message": "Login successful"


POST http://127.0.0.1:8000/app/login/
http://127.0.0.1:8000/app/login/
Save
POST
http://127.0.0.1:8000/app/login/
Send
Params
Authorization
Headers (11)
Body
Pre-request Script
Tests
Settings
Cookies
none
form-data
x-www-form-urlencoded
raw
binary
JSON
Beautify
"username":"jane",
"password": "janae"
Body
Cookies (2)
Headers (10)
Test Results
400 Bad Request
435 ms
375 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
"error": "Invalid username or password"

```python
driver = webdriver.Chrome()
driver.get("http://localhost:3000/")

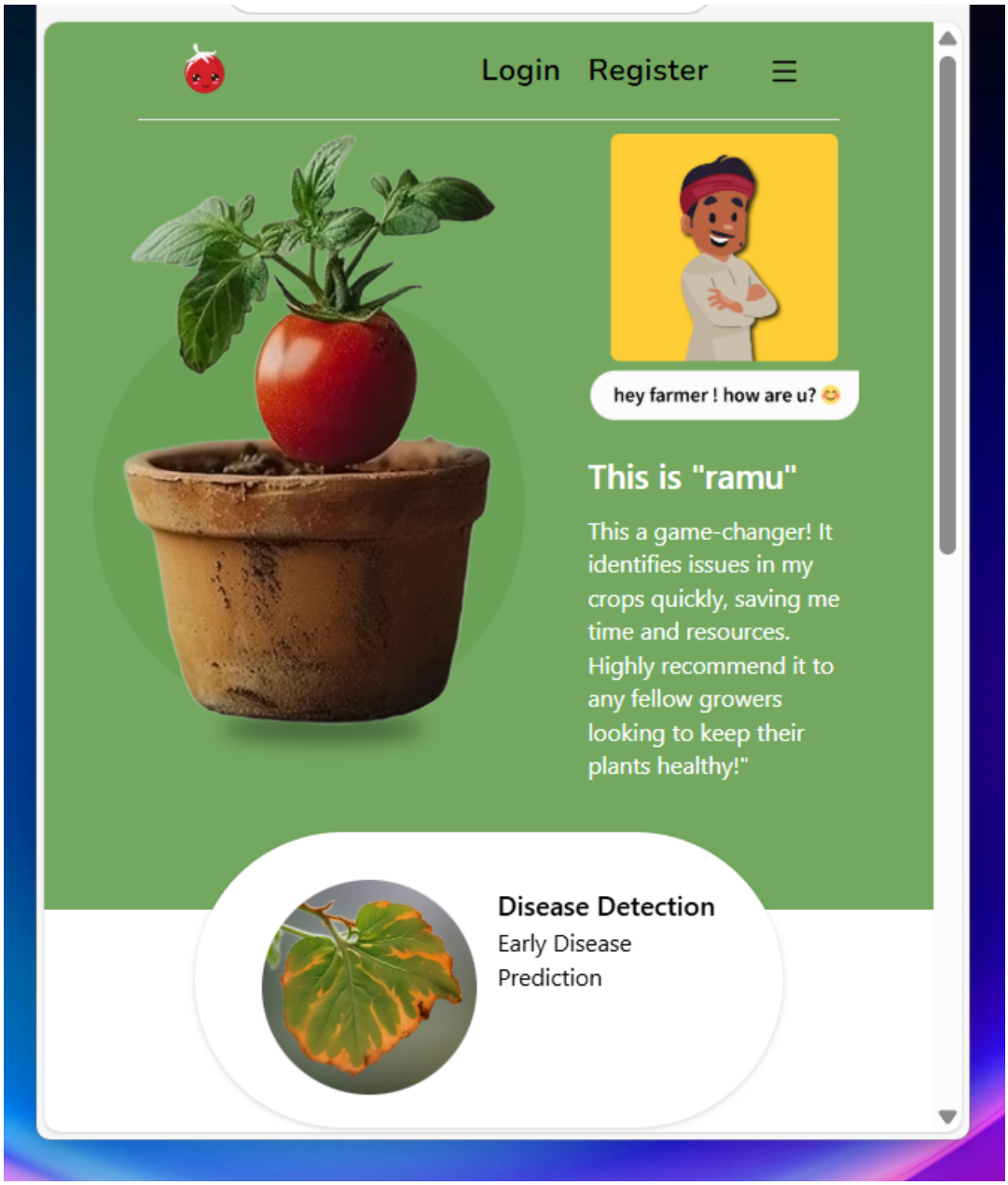
driver.maximize_window()
logen=driver.find_element(By.ID,"Loginbutton")
logen.click()
time.sleep(2)
username=driver.find_element(By.ID,"username")
username.send_keys("jane")
password=driver.find_element(By.ID,"password")
password.send_keys("jane")
login=driver.find_element(By.ID,"submitt")
login.click()
time.sleep(2)
fileo=driver.find_element(By.CSS_SELECTOR,"input[type=file]")
fileo.send_keys("C:/Users/AASHIK/Desktop/project model handeling/1/Our image samples - Copy/powdery_mildew/pm2_change_90.jpg")
time.sleep(2)
driver.find_element(By.CSS_SELECTOR,"button[type=submit]").click()
time.sleep(2)
spans= driver.find_element(By.ID,'Out_put')
p_tags = spans.find_elements(By.TAG_NAME, 'p')
# Print the text from each <p> tag
for p in p_tags:
    print(p.text)
driver.close()
```

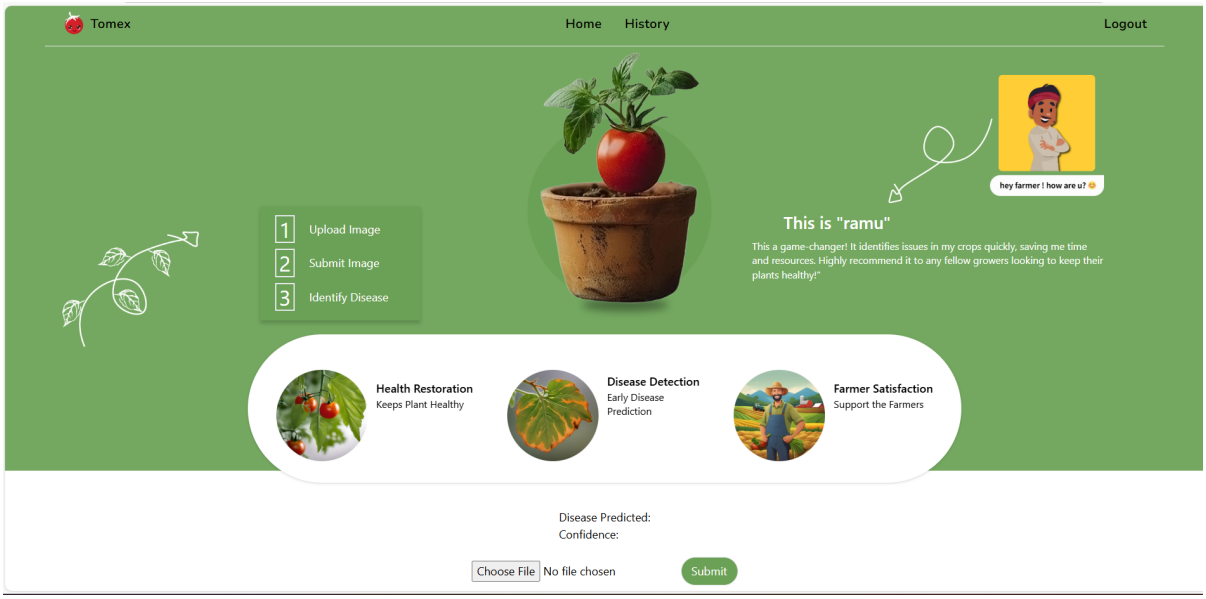
```
DevTools listening on ws://127.0.0.1:55975/devtools/browser/73f421b7-b751-4094-aefb-0b0d3b3319cc
PS C:\Users\AASHIK\Desktop\Plant_Disease_project\testing> python .\automated_testing.py

DevTools listening on ws://127.0.0.1:55975/devtools/browser/73f421b7-b751-4094-aefb-0b0d3b3319cc

DevTools listening on ws://127.0.0.1:55975/devtools/browser/73f421b7-b751-4094-aefb-0b0d3b3319cc
DevTools listening on ws://127.0.0.1:55975/devtools/browser/73f421b7-b751-4094-aefb-0b0d3b3319cc
Disease Predicted: Powdery mildew
Confidence: 63.95
PS C:\Users\AASHIK\Desktop\Plant_Disease_project\testing>
```

UI:


Login
Register
hey farmer ! how are u?
This is "ramu"
This a game-changer! It identifies issues in my crops quickly, saving me time and resources. Highly recommend it to any fellow growers looking to keep their plants healthy!"
Disease Detection
Early Disease Prediction


Tomex
Home
History
Logout
hey farmer ! how are u?
This is "ramu"
This a game-changer! It identifies issues in my crops quickly, saving me time and resources. Highly recommend it to any fellow growers looking to keep their plants healthy!"
1 Upload Image
2 Submit Image
3 Identify Disease
Health Restoration
Keeps Plant Healthy
Disease Detection
Early Disease Prediction
Farmer Satisfaction
Support the Farmers
Disease Predicted:
Confidence:
Choose File
No file chosen
Submit